Python: exercises on strings and lists

This notebook was generated in solutions mode.

Exercise: 1: Converting String to List

Exercise Description

Convert the string `s = "Python is fun!"` to a list of characters.

Your solution

```
s = "Python is fun!"
L = list(s)
print(L)
# Output: ['P', 'y', 't', 'h', 'o', 'n', ' ', 'i', 's', ' ', 'f', 'u', 'n', '!']
```

Test your solution

```
# Test the string to list conversion
s = "Python is fun!"
L = list(s)
expected = ['P', 'y', 't', 'h', 'o', 'n', ' ', 'i', 's', ' ', 'f', 'u', 'n', '!']
assert L == expected, f"Expected {expected}, got {L}"
print("✓ Test passed: String converted to list correctly")
print(f"Result: {L}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 2: Splitting a String by Whitespace

Exercise Description

Split the string `s = "I love programming"` into a list of words.

Your solution

```
s = "I love programming"
L = s.split()
print(L)
# Output: ['I', 'love', 'programming']
```

Test your solution

```
# Test string splitting by whitespace
s = "I love programming"
L = s.split()
expected = ['I', 'love', 'programming']
assert L == expected, f"Expected {expected}, got {L}"
print("✓ Test passed: String split by whitespace correctly")
print(f"Result: {L}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 3: Custom Character Split

Exercise Description

Split the string `s = "apple-banana-orange"` using `-` as the delimiter.

Your solution

```
s = "apple-banana-orange"
L = s.split("-")
print(L)
# Output: ['apple', 'banana', 'orange']
```

Test your solution

```
# Test string splitting by custom delimiter
s = "apple-banana-orange"
L = s.split("-")
expected = ['apple', 'banana', 'orange']
assert L == expected, f"Expected {expected}, got {L}"
print("✓ Test passed: String split by delimiter correctly")
print(f"Result: {L}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the

Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 4: Joining a List into a String

Exercise Description

Join the list `L = ['Hello', 'World']` into a string without spaces.

Your solution

```
L = ["Hello", "World"]
s = "".join(L)
print(s)
# Output: 'HelloWorld'
```

Test your solution

```
# Test joining list without spaces
L = ['Hello', 'World']
result = ''.join(L)
expected = 'HelloWorld'
assert result == expected, f"Expected {expected}, got {result}"
```

```
print("✓ Test passed: List joined without spaces correctly")
print(f"Result: {result}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 5: Joining with a Custom Character

Exercise Description

Join the list `L = ['data', 'science']` into a string with an underscore `_` between words.

Your solution

```
L = ["data", "science"]
s = "_".join(L)
print(s)
# Output: 'data_science'
```

Test your solution

```
# Test joining list with underscore
L = ['data', 'science']
result = '_'.join(L)
expected = 'data_science'
assert result == expected, f"Expected {expected}, got {result}"
print("✓ Test passed: List joined with underscore correctly")
print(f"Result: {result}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 6: Sorting a List

Exercise Description

Sort the list `L = [8, 3, 5, 2]` in ascending order using `sort()`.

Your solution

```
L = [8, 3, 5, 2]
L.sort()
print(L)
# Output: [2, 3, 5, 8]
```

Test your solution

```
# Test list sorting in place
L = [8, 3, 5, 2]
L.sort()
expected = [2, 3, 5, 8]
assert L == expected, f"Expected {expected}, got {L}"
print("✓ Test passed: List sorted correctly")
print(f"Result: {L}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 7: Reversing a List

Exercise Description

Reverse the list `L = [1, 2, 3, 4]`.

Your solution

```
L = [1, 2, 3, 4]
L1 = L[::-1]
L.reverse()
print(L)
print(L1)
# Output: [4, 3, 2, 1]
```

Test your solution

```
# Test list reversal in place
L = [1, 2, 3, 4]
L.reverse()
expected = [4, 3, 2, 1]
assert L == expected, f"Expected {expected}, got {L}"
print("✓ Test passed: List reversed correctly")
print(f"Result: {L}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 8: Using `sorted()` to Sort a List

Exercise Description

Use `sorted()` to sort the list `L = [10, 5, 8, 3]` without modifying the original list.

Your solution

```
L = [10, 5, 8, 3]
sorted_L = sorted(L)
print(sorted_L)
# Output: [3, 5, 8, 10]
```

Test your solution

```
# Test sorted() function
L = [10, 5, 8, 3]
original = L.copy()
sorted_L = sorted(L)
```

```
expected_sorted = [3, 5, 8, 10]
assert sorted_L == expected_sorted, f"Expected {expected_sorted}, got {sorted_L}"
assert L == original, f"Original list should be unchanged, got {L}"
print("✓ Test passed: sorted() works correctly")
print(f"Original: {L}")
print(f"Sorted: {sorted_L}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 9: Aliasing a List

Exercise Description

Create a list `a = [1, 2, 3]`, and assign `b = a`. Modify `b` and show how it affects `a`.

Your solution

```
def aliasing_a_list(lst):
    a = [1, 2, 3]
```

```
    b = a
    b[0] = 99
    print(a)
    return a
```

Test your solution

```
# Test the exercise
print("✓ Exercise completed successfully")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 10: Cloning a List

Exercise Description

Clone the list `a = [4, 5, 6]` to a new list `b` and modify `b` without affecting `a`.

Your solution

```
a = [4, 5, 6]
b = a[:]
b[0] = 10
print(a)  # Output: [4, 5, 6]
print(b)  # Output: [10, 5, 6]
```

Test your solution

```
# Test the exercise
print("✓ Exercise completed successfully")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 11: Nested Lists and Aliasing

Exercise Description

Create a nested list `colors = [warm]`, where `warm = ['yellow', 'orange']`. Add `hot = ['red']` to the nested list. Show how appending to `hot` affects `colors`.

Your solution

```
warm = ["yellow", "orange"]
hot = ["red"]
colors = [warm]
colors.append(hot)
hot.append("pink")
print(colors)
# Output: [['yellow', 'orange'], ['red', 'pink']]
```

Test your solution

```
# Test the exercise
print("✓ Exercise completed successfully")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 12: Mutating a List While Iterating

Exercise Description

Explain why the following code is problematic. Suggest a fix:

Your solution

```
def remove_dups(L1, L2):
    L1_copy = L1[:]
    for item in L1_copy:
        if item in L2:
            L1.remove(item)
    return L1
```

Test your solution

```
# Test the exercise
print("✓ Exercise completed successfully")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 13: Removing Duplicates

Exercise Description

Write a function `remove_dups(L1, L2)` that removes all elements from `L1` that are also in `L2`.

Your solution

```
def remove_dups(L1, L2):
    L1_copy = L1[:]
    for item in L1_copy:
        if item in L2:
            L1.remove(item)
    return L1


L1 = [1, 2, 3, 4]
L2 = [1, 2, 5, 6]
print(remove_dups(L1, L2))
# Output: [3, 4]
```

Test your solution

```python
# Test remove_dups function
L1 = [1, 2, 3, 4, 5]
L2 = [2, 4]
remove_dups(L1, L2)
expected = [1, 3, 5]
assert L1 == expected, f"Expected {expected}, got {L1}"
print("✓ Test passed: Duplicates removed correctly")
print(f"Result: {L1}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 14: List Mutation and Sorting

Exercise Description

Explain the difference between using `sort()` and `sorted()` on the list `L = [5, 3, 8]`.

Your solution

```
L = [5, 3, 8]
sorted_L = sorted(L)  # Doesn't modify L, returns a new list
print(L)  # Output: [5, 3, 8]
L.sort()  # Modifies L in place
print(L)  # Output: [3, 5, 8]
```

Test your solution

```
# Test the exercise
print("✓ Exercise completed successfully")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 15: Using List Methods in a Loop

Exercise Description

Use a loop to sort and reverse the list `L = [7, 2, 9]`. Apply the transformations two times.

Your solution

```
L = [7, 2, 9]
for _ in range(2):
    L.sort()  # Sorts the list
    L.reverse()  # Reverses the sorted list
print(L)
# Output: [9, 7, 2]
```

Test your solution

```
# Test the exercise
print("✓ Exercise completed successfully")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 16: Reconstruct String from Substrings

Exercise Description

You are given a string `s = "data_science_is_fun"`. Split it into individual words and then reconstruct the string by joining the words with a hyphen `-`, but in reverse order.

Your solution

```
s = "data_science_is_fun"
words = s.split('_')
reversed_s = '-'.join(words[::-1])
print(reversed_s)
# Output: 'fun-is-science-data'
```

Test your solution

```
# Test the exercise
print("✓ Exercise completed successfully")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 17: Alternating Split and Join

Exercise Description

Given the string `s = "The_quick_brown_fox"`, split the string at underscores `_` and then join the resulting list using spaces `' '`. Next, reverse the string and split it by spaces again.

Your solution

```
s = "The_quick_brown_fox"
split_s = s.split('_')
joined_s = ' '.join(split_s)
reversed_s = joined_s[::-1]
reversed_split = reversed_s.split()
print(reversed_split)
# Output: ['xof', 'nworb', 'kciuq', 'ehT']
```

Test your solution

```
# Test the exercise
print("✓ Exercise completed successfully")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 18: List Mutation Inside a Function

Exercise Description

Write a function `append_and_sort(L)` that takes a list `L`, appends a given value (e.g., 10), sorts the list, and returns it. Make sure the original list is not mutated inside the function.

Your solution

```
def append_and_sort(L):
    L_copy = L[:]  # Clone the list to avoid mutation
    L_copy.append(10)
    L_copy.sort()
    return L_copy

L = [4, 2, 8, 5]
```

```
print(append_and_sort(L))  # Output: [2, 4, 5, 8, 10]
print(L)  # Output: [4, 2, 8, 5]
```

Test your solution

```
# Test append_and_sort function
L = [3, 1, 4]
original = L.copy()
result = append_and_sort(L)
expected = [1, 3, 4, 10]
assert result == expected, f"Expected {expected}, got {result}"
assert L == original, f"Original list should be unchanged, got {L}"
print("✓ Test passed: Function works without mutating original")
print(f"Original: {L}")
print(f"Result: {result}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 19: Mutating Lists Based on Conditions

Exercise Description

Write a function `remove_elements_above(L, n)` that removes all elements greater than `n` from the list `L`. Make sure to handle list mutation properly inside a loop.

Your solution

```
def remove_elements_above(L, n):
    L_copy = L[:]
    for item in L_copy:
        if item > n:
            L.remove(item)
    return L

L = [1, 3, 5, 7, 9]
print(remove_elements_above(L, 5))
# Output: [1, 3, 5]
```

Test your solution

```
# Test remove_elements_above function
L = [1, 5, 3, 8, 2, 7]
remove_elements_above(L, 5)
expected = [1, 5, 3, 2]
assert L == expected, f"Expected {expected}, got {L}"
print("✓ Test passed: Elements above threshold removed correctly")
print(f"Result: {L}")
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise: 20: Aliasing and Function Modification

Exercise Description

Create a list `a = [10, 20, 30]` and an alias `b = a`. Write a function `modify_list(L)` that appends the value `100` to `L`. Check if both `a` and `b` are modified when you pass `a` to the function.

Your solution

```
def modify_list(L):
    L.append(100)

a = [10, 20, 30]
b = a
modify_list(a)
```